

2023 Paper

Question 1

Question 2

Question 3

Question 1

(i) Two E-MOSFETS are connected in series across a 10V supply. The ‘upper’ MOSFET has its drain connected directly to its gate. The ‘lower’ MOSFET is unbiased. The input to the circuit is taken to be the gate of the lower MOSFET and the output is taken to be at the drain of the lower MOSFET.

The upper MOSFET has a resistance of 2KΩ when its gate-source voltage is 10V. The lower MOSFET has a resistance of 150Ω when the input voltage is 3V. What is the output of the circuit when the input to the circuit is 0V and 3V respectively?

Attempt:

To analyse the given circuit, consider the operation of each MOSFET separately and then combine their effects:

1. Upper MOSFET:
    a. The upper MOSFET has a resistance of 2k Ohms when its gate source voltage is 10V
    b. Since the drain is connected directly to the gate, the upper MOSFET operates in the saturation region when the input voltage is 10V (maximum gate source voltage)
2. Lower MOSFET:
    a. The lower MOSFET is unbiased initially.
    b. It has a resistance of 150 Ohm when the input voltage is 3V.

Now, we consider the 2 cases:

case 1: 0V

- the lower MOSFET is unbiased, so its resistance is not affecting the circuit.
- the upper MOSFET is in saturation, acting as a low-resistance path
- the output voltage would be the 10V since the lower MOSFET is not conducting.

case 2: 3V

- the lower MOSFET now has gate source of 3V, and it becomes biased.
- the resistance of the lower MOSFET is now 150 Ohms

- the upper MOSFET is still in saturation since its gate source voltage is below 10V
- the output voltage would be determined by the voltage division between the upper and lower MOSFET resistances.

$$R_{total} = R_{upper} + R_{lower}$$
$$R_{total} = 2000\Omega + 150\Omega = 2150\Omega$$

The output voltage is given by the voltage division formula:

$$V_{out} = V_{supply} * \frac{R_{lower}}{R_{total}}$$
$$V_{out} = 10V * \frac{150\Omega}{2150\Omega} = \frac{30}{43}V = 0.6977V$$

(ii) Consider a simple amplifier circuit. Sometimes a small resistor is connected from the source to ground. This has the effect of stabilising the output where the supply voltage fluctuates. Explain how this works.

Attempt:

The additional of a small resistor from the source to ground in a simple amplifier circuit can serve as a form of negative feedback and helps to stabilise the output, especially when the supply voltage fluctuates. This technique is often referred to as emitter degeneration in the context of transistor amplifiers.

Here is how it works:

1. Emitter Degeneration
    a. In a transistor amplifier, the emitter resistor (connected from the emitter to the ground) is the small resistor
    b. This resistor introduces negative feedback, as a portion of the emitter current flows through it
2. Stabilising Effect:
    a. When the supply voltage fluctuates, the current flowing through the transistor can change
    b. The emitter resistor helps to decouple the collector current from the variations in the transistor's characteristics (like its beta value)
    c. As the emitter resistor absorbs some of the variations in the transistor's parameters, it provides a stabilising effect on the output.
3. Current Stability:
    a. The emitter resistor converts changes in collector current into changes in voltage across itself according to Ohm's law (V = IR)
    b. This voltage drop across the emitter resistor effectively stabilises the bias point of the transistor.
4. Reducing Sensitivity:
    a. The emitter resistor reduces the sensitivity of the amplifier to changes in transistor parameters, making the circuit more robust against fluctuations in the supply voltage and transistor characteristics
5. Improved Linearity:
    a. The negative feedback introduced by the emitter resistor helps improve the linearity of the amplifier. It reduces distortion and makes the amplifier response more linear over a range of input signals
6. Thermal Stability:
    a. The emitter resistor also contributes to thermal stability. As the temperature of the transistor changes, the emitter resistor helps maintain a more stable bias point.

Question 2

Design a four-bit combinational circuit 2's complementer with inputs ABCD and outputs wxyz. The output generates the 2's complement of the input binary number. Write down the truth table, karnaugh maps and boolean equations for the circuit.

Attempt:

Truth Table

A	B	C	D	w	x	y	z
0	0	0	0	0	0	0	0
0	0	0	1	1	1	1	1
0	0	1	0	1	1	1	0
0	0	1	1	1	1	0	1
0	1	0	0	1	1	0	0
0	1	0	1	1	0	1	1
0	1	1	0	1	0	1	0
0	1	1	1	1	0	0	1
1	0	0	0	1	0	0	0
1	0	0	1	0	1	1	1
1	0	1	0	0	1	1	0
1	0	1	1	0	1	0	1
1	1	0	0	0	1	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	0	1	0
1	1	1	1	0	0	0	1

K maps:

for w:

\	CD	00	01	11	10
AB	|	-	-	-	-
00	|	0	1	1	1
01	|	1	1	1	1
11	|	0	0	0	0
10	|	1	0	0	0

I got: AB’C’D’ + A’D+ A’CD’ + A’BC’D’

The answer: A XOR (B + C + D)

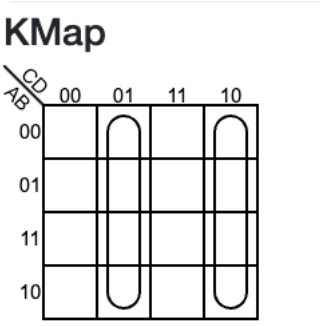
for x:

\	CD	00	01	11	10
AB	|	-	-	-	-
00	|	0	1	1	1
01	|	1	0	0	0
11	|	1	0	0	0
10	|	0	1	1	1

I got: B’C + BC’D’ + B’C’D = BC’D’ + B’C + B’D

The answer: B XOR (C + D)

For y:



I got: C'D+ CD'

Answer: C XOR D

https://www.boolean-algebra.com/kmap?q=4-11+1_110+1_11+11_110+11_11+110_110+110_11+10_110+10

For z:

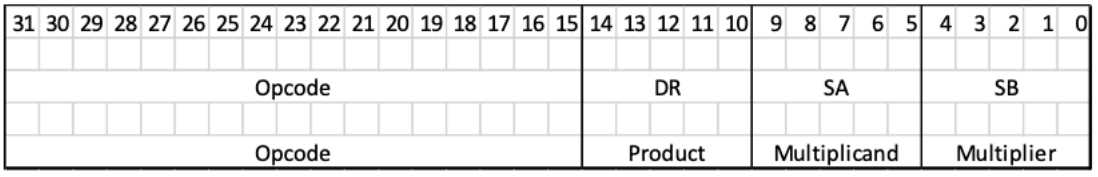
https://www.boolean-algebra.com/kmap?q=4-11+1_10+1_10+11_11+11_10+10_11+10_11+110_10+110

Question 3

Design an Algorithm State Machines (ASM) for a multiplication instruction that can be implemented on the processor that you designed and simulated as part of your coursework for the "CSU22022 - Computer Architecture I" module. The processor architecture is shown in Figure 6. The ASM should implement a **Two's Complement Multiplication**. Product (32 bit) = Multiplicand (16 bit) x Multiplier (16 bit).

1. You should provide a ASM chart for the two's complement multiplication and a written description for the various states and the state transitions of the ASM. This should include all register transfer operations that need to be executed.
2. You should also populate the first 15 consecutive Control Memory Addresses that implement the first 15 states of your ASM. Therefore, showing how your ASM design is implemented on the processor Figure 6. A Control Memory template is shown in Figure 7.

   Please assume the following:
   The Two's Complement Multiplication Instruction has the following format Figure 1:



DR – is the address of the register that will receive the output of your two's complement multiplication. The Product should be in a 32 bit two's complement format.
SA - is the address of the register that holds the 16 bit Multiplicand in a two's complement format. The 16 bit Multiplicand is sign extended to 32 bit.
SB - is the address of the register that holds the 16 bit Multiplier in a two's complement format. The 16 bit Multiplier is sign extended to 32 bit.

The Multiplicand and the Multiplier can be positive or negative. Therefore, your ASM design needs to be able to compute the four possible combinations.

Figure 2 shows an example of an unsigned multiplication with partial product. This is how we implemented the binary multiplication in class.

Decimal	Binary	Unsigned Multiplication with Partial product
10	1010	multiplicand
x 11	x 1011	multiplier
	0000 0000	partial product
	+ 0000 1010	shifted multiplicand
	0000 1010	partial product
	+ 0001 0100	shifted multiplicand
	0001 1110	partial product
	+ 0000 0000	shifted multiplicand
	0001 1110	partial product
	+ 0010 1000	shifted multiplicand
= 110	= 0111 1110	**Product**

When implementing a binary 2's complement multiplication, we must consider a **positive multiplier** and a **negative multiplier**. Figure 3 describes a 2's Complement Multiplication with negative multiplier.

Decimal	Binary	2's Complement Multiplication with negative multiplier
- 6	1111 1010	multiplicand (sign extended)
X - 5	x 1011	multiplier (MSB has a negative weight)
	0000 0000	partial product
	+ 1111 1010	shifted multiplicand
	1111 1010	partial product
	+ 1111 0100	shifted multiplicand
	1110 1110	partial product
	+ 0000 0000	shifted multiplicand
	1110 1110	partial product
	+ 0010 1000	shifted and negated multiplicand
= 30	= 0001 1110	**product**
	1010	multiplicand
	0101	1's complement
	0110	2's complement

A 2's complement multiplication with a positive multiplier works just like an unsigned multiplication. Figure 4 depicts a 2's Complement Multiplication with positive multiplier.

Decimal	Binary	2's Complement Multiplication with positive multiplier
- 6	1111 1010	multiplicand (sign extended)
x 5	x 0101	multiplier (MSB has a zero weight)
	0000 0000	partial product
	+ 1111 1010	shifted multiplicand
	1111 1010	partial product
	+ 0000 0000	shifted multiplicand
	1111 1010	partial product
	+ 1110 1000	shifted multiplicand
	1110 0010	partial product
	+ 0000 0000	shifted multiplicand, zero weight
= - 30	= 1110 0010	**product**

The processor Figure 6 implements the functions shown in Figure 5. The F = srB and F slB functions perform a single bit shift. They insert a '0' in the vacated MSB or LSB position. Furthermore, these shift operations set the carry flag if the pushed-out bit is a one.

FS Code												
FS 4	FS 3	FS 2	FS 1	FS 0	MF Select	G Select				H Select		Micro-operation
S3	S2	S1	S0	C	S3	S2	S1	S0	C	S2	S1	
0	0	0	0	0	0	0	0	0	0	0	0	F = A
0	0	0	0	1	0	0	0	0	1	0	0	F = A + 1
0	0	0	1	0	0	0	0	1	0	0	0	F = A + B
0	0	0	1	1	0	0	0	1	1	0	0	F = A + B + 1
0	0	1	0	0	0	0	1	0	0	0	1	F = A + NOT(B)
0	0	1	0	1	0	0	1	0	1	0	1	F = A + NOT(B) + 1
0	0	1	1	0	0	0	1	1	0	0	1	F = A - 1
0	0	1	1	1	0	0	1	1	1	0	1	F = A
0	1	0	0	0	0	1	0	0	0	1	0	F = A AND B
0	1	0	1	0	0	1	0	1	0	1	0	F = A OR B
0	1	1	0	0	0	1	1	0	0	1	1	F = A XOR B
0	1	1	1	0	0	1	1	1	0	1	1	F = NOT(A)
1	0	0	0	0	1	0	0	0	0	0	0	F = B
1	0	1	0	0	1	0	1	0	0	0	1	F = srB
1	1	0	0	0	1	1	0	0	0	1	0	F = slB

```
architecture Behavioral of CPU_ControlMemory_XXXXXXXX is

-- we use the least significant 7 bit of the Address - array(0 to 127)

type ROM_array is array(0 to 127) of STD_LOGIC_VECTOR (50 downto 0);

signal ROM : ROM_array :=(

--|50            34|33  31|30| 29| 28| 27|26  23|22  19|18  15| 14|13   09| 08| 07| 06| 05| 04| 03| 02| 01| 00|  Memory
--|  Next Address   |  MS  |MC| IL| PI| PL|  TD  |  TA  |  TB  | MB|   FS  | MD| RW| MM| MW| RV| RC| RN| RZ| FL|  Address
  "00000000000000000"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 00
  "00000000000000001"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 01
  "00000000000000010"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 02
  "00000000000000011"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 03
  "00000000000000100"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 04
  "00000000000000101"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 05
  "00000000000000110"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 06
  "00000000000000111"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 07
  "00000000000001000"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 08
  "00000000000001001"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 09
  "00000000000001010"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 0A
  "00000000000001011"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 0B
  "00000000000001100"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 0C
  "00000000000001101"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 0D
  "00000000000001110"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 0E
  "00000000000001111"&"000"&'0'&'0'&'0'&'0'&'0'&"0000"&"0000"&"0000"&'0'&"00000"&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0'&'0', -- 0F
```

Answer:

Algorithm State Machines (ASM) for Two's Complement Multiplication:

To implement a Two's Complement Multiplication instruction, we will require a few states. Here is an outline of the states and transitions:

1. **Initialisation State (S0):** In this state, the multiplicand (M) is loaded into a register (say, R1), the multiplier (Q) is loaded into another register (say, R2), and a product register (P) is initialized to 0. The counter (n) is set to the number of bits in the multiplier.

2. **Multiplication State (S1):** This state consists of the core multiplication operation. If the least significant bit (LSB) of the multiplier Q (i.e., Q[0]) is 1, then the multiplicand M is added to the product P. This addition is performed in a two's complement system.
3. **Shift Right State (S2):** The result in P is shifted one bit to the right, effectively halving the value.
4. **Decrement Counter State (S3):** The counter n is decremented by 1. If the counter is not zero, the control is transferred back to the Multiplication State (S1). If the counter is zero, the multiplication operation is complete, and the control is transferred to the End State (S4).
5. **End State (S4):** In this state, the final product in the product register P is transferred to the destination register (DR) specified in the instruction.

The transitions are conditional based on the LSB of Q and the value of the counter n.

Register Transfer Operations:

1. Initialisation: Load M -> R1, Load Q -> R2, Load 0 -> P, n = 16
2. Multiplication: If Q[0] = 1, then P = P + M
3. Shift Right: P = P >> 1, Q = Q >> 1 (with Q[n] receiving the most significant bit of P)
4. Decrement Counter: n = n - 1
5. End: If n = 0, then DR = P

Implementation on Processor:

The implementation of the ASM on the processor would require translating each of these states and transitions into a sequence of micro-operations that the processor can execute. The control memory addresses will be populated with the microcodes corresponding to the micro-operations for each state. This would include operations like Load, Add, Shift Right, Decrement, and Store.